Dhad Readability Classifier

Take-home report: multi-label difficulty scoring, built with an agent-assisted, evaluation-first workflow

Bisharah Estephan · Miqyas al Dhad contractor application · 29 October 2026

0.805
val macro F1 (floor 0.524)

+0.65
encoder response to unseen technical words

20
commits in 86 minutes

13
AI agents for research, review, build and QC

At a glance

A model that, given a short English passage, outputs three independent probabilities: lexical load, syntactic complexity and conceptual abstraction. The delivered TF-IDF + logistic regression model scores macro F1 0.805 (95% CI 0.75–0.86) on the held-out validation set, against 0.524 for always predicting 'present'. Validation data was opened exactly once, after the model and thresholds were frozen under the git tag pre-val.

The central finding is about unseen vocabulary. In this data each label is driven by a handful of slot words, so ordinary cross-validation rewards memorisation. Holding whole words out drops the baseline from 0.828 to 0.741; a fine-tuned DeBERTa-v3 encoder reaches 0.772. Controlled behavioural tests show why: shown words that never occur in training, the baseline barely reacts while the encoder responds almost as strongly as to familiar words.

1. The project

The brief asked for a multi-label classifier over a synthetic dataset (480 training and 120 validation passages) within a two-hour budget, using local or free compute only, and without hand-written rules derived from the validation set. It evaluates ways of working and motivated technical choices, not raw model quality.

The brief asked for	What was delivered
Training code that reads train.csv and writes a checkpoint	**uv run dhad-train**: cross-validation, threshold tuning, refit, checkpoint and run manifest
Three probabilities per passage (multi-label)	predict_proba returns an (n, 3) array in [0, 1]
Evaluation on val.csv with per-dimension and macro F1	**uv run dhad-evaluate**: macro F1 0.805 with bootstrap CIs, plus ROC-AUC, average precision, Brier and a shortcut check
README of one page or less with 2–3 design choices	README with three design choices, results and limitations
Local compute, no rules written from val	Mac + personal RTX 5090; val read by one module, once

Beyond the brief: a DeBERTa-v3 encoder, held-out-vocabulary cross-validation, behavioural tests, 20 unit tests, a reproducible train-only data analysis, and a fresh-clone check that reproduces every committed number exactly.

2. How I worked

The work ran as an evaluation-first pipeline. I set the rules first, then used AI agents for breadth (research, review, parallel implementation, QC), and kept every decision that touches evaluation under

my own control.

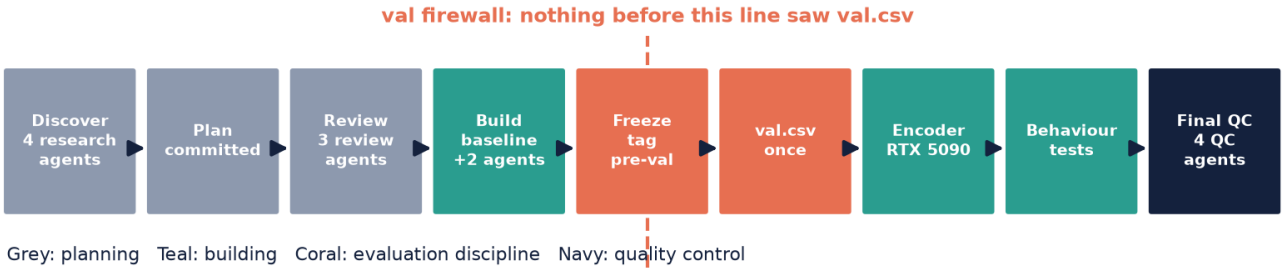


Figure 1. Delivery pipeline. The dashed line is the validation firewall.

Thirteen agents, each with a narrow brief

Research (4): modelling, evaluation methodology, engineering stack and a train-only data analysis. Plan review (3): model choice and licence check against Hugging Face, a data-scientist and readability-expert critique, and a grader's view. Build (2): unit tests and the analysis script, written in parallel on files that did not overlap. Final QC (4, read-only): every number against the result files, code correctness, privacy and git hygiene, and the brief's deliverables.

Agent output was checked, not trusted: the reviews reordered the plan (baseline deliverables first, encoder time-boxed), one agent's claim of swapped figures was checked and rejected, and QC findings were applied only when verified. Correct but untestable code issues were documented rather than patched blind.

Delivery timeline

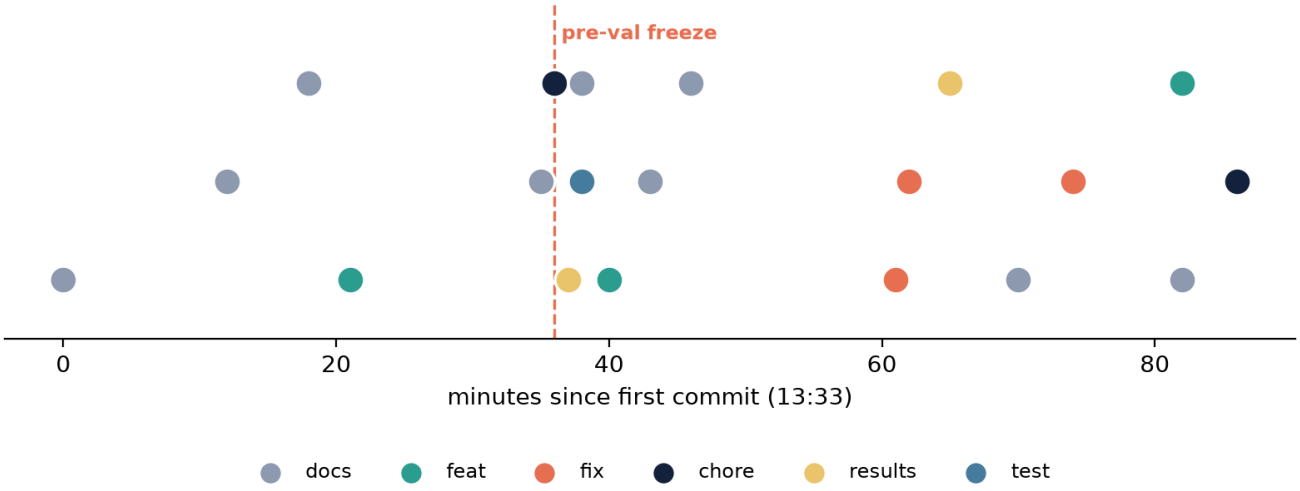


Figure 2. 20 commits in 86 minutes, coloured by conventional-commit type. The freeze precedes the single val run.

3. Results

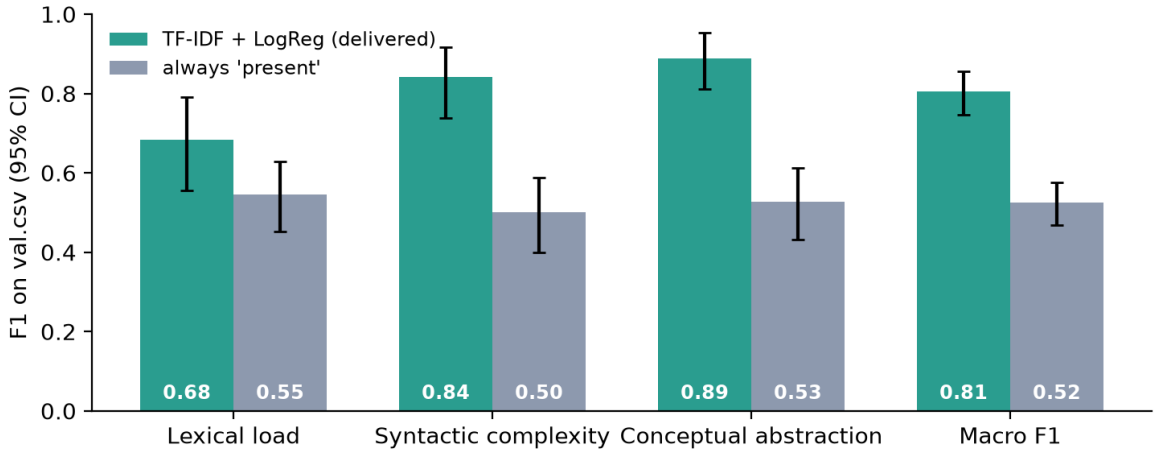


Figure 3. Validation F1 with 95% bootstrap CIs (2,000 resamples), computed once, after the freeze.

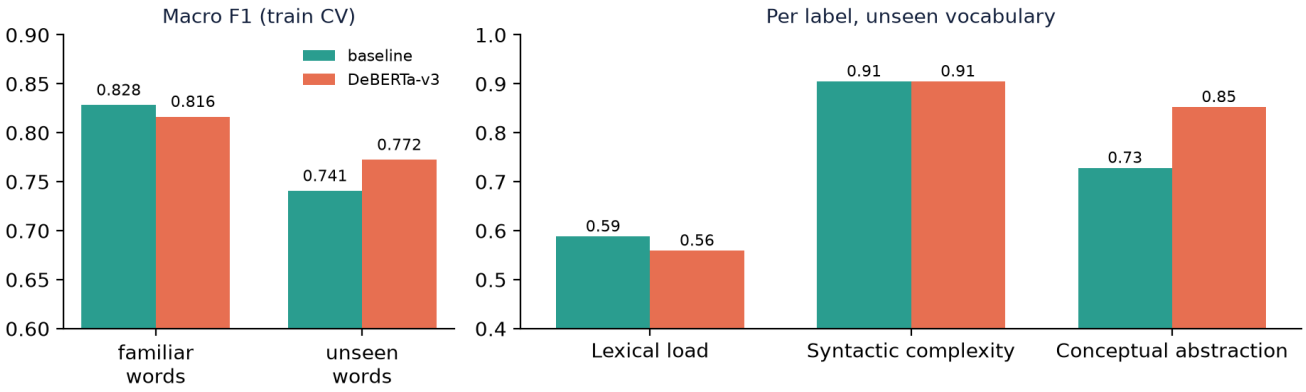


Figure 4. Train cross-validation. On unseen words the encoder holds up; the gain is in abstraction. Lexical load does not improve.

What did each model actually learn?

F1 cannot show why a model is right. The behavioural tests change one factor at a time in 200 minimal pairs built from the training templates, using words seen in training and words verified never to occur in it (e.g. isotope, jurisprudence, morality, virtue). Validation data is not used.

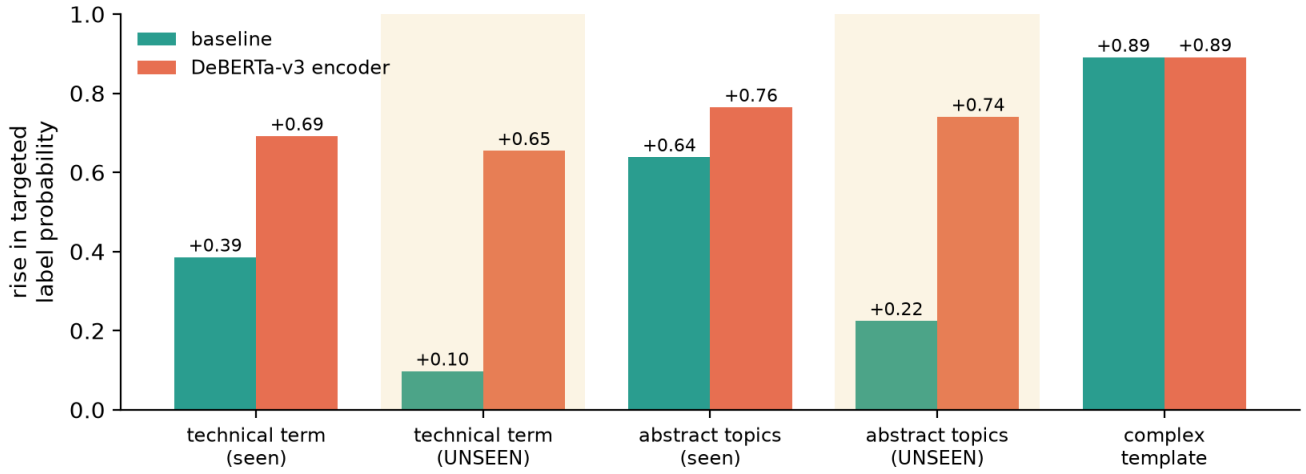


Figure 5. Rise in the targeted label's probability. Highlighted: unseen words, where only the encoder generalises. Off-target shifts stay at 0.04 or less, except a shared shortcut: the complex template alone raises lexical load (+0.10 / +0.17).

4. Tools

Area	Tools	Why
Coding agent	Claude Code with parallel sub-agents	Breadth for research, review, build and QC; decisions kept with me
Environment	uv, uv.lock, Python 3.12+	Reproducible installs; heavy deep-learning stack is an optional extra
Modelling	scikit-learn, PyTorch 2.15, transformers 5.21	TF-IDF + LR baseline; DeBERTa-v3-base (MIT) fine-tune
Evaluation	iterative-stratification, bootstrap CIs	Multi-label stratified and held-out-vocabulary CV
Quality	pytest (20 tests), ruff lint + format	Fast tests with no downloads
Compute	Mac for development, personal RTX 5090 for the encoder	Local only, no paid APIs
Reporting	matplotlib, reportlab	This report is generated from the result files

5. Governance in the GitHub repository



Figure 6. Rules flow from AGENTS.md through the agents to every pushed commit.

Mechanism	What it guarantees
AGENTS.md (checked in)	Hard rules for any coding agent: val never used for training, tuning or selection; every number from a real run; no secrets or large files; no history rewriting; ask when unsure.
pre-val annotated tag	Model and thresholds frozen in results/selection.json before val.csv was read. The val results commit comes after the tag.
Single val reader	Only dhad-evaluate reads val.csv; tests and the behavioural script never do.
post-commit hook	Every commit is pushed immediately; failures go to .git/push.log.
Conventional commits	feat / fix / test / docs / results / chore, with Co-Authored-By trailers that make agent involvement visible.
Run manifests	Each run records its git SHA, the SHA-256 of the data and the chosen settings.
Plan and progress in docs/	Research, plan, review revisions and a story-point tracker are committed, so the process can be audited.

6. Honest limitations

• Syntactic complexity is learned from the template: F1 is 0.00 within the simple template and 0.94 within the complex one.

• Labels are synthetic and noisy, which caps the reachable F1 (ceilings in results/train_analysis.md).
• 120 validation rows: differences under about 0.05 F1 are noise.
• The encoder was trained after the freeze, so it is exploratory and has no val score. Two bugs found on the way (fp16 weights under transformers 5, and a config file overwrite) were fixed and documented.

Next steps and transfer to Arabic

Swap in an Arabic encoder (CAMeLBERT-MSA, AraBERTv2) and turn the held-out-vocabulary CV into a held-out-root CV; measure lexical load from root and pattern rarity, and syntax from clitic and iḍāfa depth; move from binary flags to human-labelled ordinal difficulty levels.